

Practical 03

Q

uestion

Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

```
GNU nano 7.2
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    pid_t pid;

    printf("Process started\n");

    pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
    }
    else if (pid == 0) {
        printf("Child Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());
        printf("Child is running\n");

        sleep(5);

        printf("Child process terminated\n");
    }
    else {
        printf("Parent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Parent is running\n");

        sleep(10);

        printf("Parent process terminated\n");
    }

    return 0;
}
```

```

teja@LAPTOP-FGDPB110:~$ nano week3_task3.c
teja@LAPTOP-FGDPB110:~$ gcc week3_task3.c -o week3_task3
teja@LAPTOP-FGDPB110:~$ ./week3_task3
Process started
Parent Process
Parent PID: 831
Parent is running
Child Process
Child PID: 832
Parent PID: 831
Child is running
Child process terminated
Parent process terminated
teja@LAPTOP-FGDPB110:~$ ps
  PID TTY          TIME CMD
 705 pts/0    00:00:00 bash
 833 pts/0    00:00:00 ps
teja@LAPTOP-FGDPB110:~$ ps -ef
UID          PID    PPID  C STIME TTY          TIME CMD
root         1        0  0 13:31 ?        00:00:01 /sbin/init
root         2        1  0 13:31 hvc0     00:00:00 /init
root         6        2  0 13:31 hvc0     00:00:00 plan9 --control-socket 7 --log-level 4 --server-fd 8 --pipe-fd 10 --log-truncate
root        58        1  0 13:31 ?        00:00:00 /usr/lib/systemd/systemd-journald
root       109        1  0 13:31 ?        00:00:01 /usr/lib/systemd/systemd-udev
systemd+   123        1  0 13:31 ?        00:00:00 /usr/lib/systemd/systemd-resolved
systemd+   124        1  0 13:31 ?        00:00:00 /usr/lib/systemd/systemd-timesyncd
root       187        1  0 13:31 ?        00:00:00 /usr/sbin/cron -f -p
message+   188        1  0 13:31 ?        00:00:00 @dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation --syslog-only
root       202        1  0 13:31 ?        00:00:00 /usr/lib/systemd/systemd-logind
root       226        1  0 13:31 tty1     00:00:00 /sbin/agetty -o -p -- \u --noclear - linux
syslog     227        1  0 13:31 ?        00:00:00 /usr/sbin/rsyslogd -n -iNONE
root       234        1  0 13:31 ?        00:00:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal
root       340        2  0 13:32 pts/1   00:00:00 /bin/login -f
teja       390        1  0 13:32 ?        00:00:00 /usr/lib/systemd/systemd --user
teja       391        390  0 13:32 ?        00:00:00 (sd-pam)
teja       415       340  0 13:32 pts/1   00:00:00 -bash
root       700        2  0 13:50 ?        00:00:00 /init
root       702       700  0 13:50 ?        00:00:00 /init
teja       705       702  0 13:50 pts/0    00:00:00 -bash
root       793        1  0 14:03 ?        00:00:00 /usr/libexec/wsl-pro-service
root      834       109  0 14:06 ?        00:00:00 (udev-worker)

```

```

root       700        2  0 13:50 ?        00:00:00 /init
root       702       700  0 13:50 ?        00:00:00 /init
teja       705       702  0 13:50 pts/0    00:00:00 -bash
root       793        1  0 14:03 ?        00:00:00 /usr/libexec/wsl-pro-service
root      834       109  0 14:06 ?        00:00:00 (udev-worker)
root      835       109  0 14:06 ?        00:00:00 (udev-worker)
teja      836       705  0 14:06 pts/0    00:00:00 ps -ef
teja@LAPTOP-FGDPB110:~$ top
top - 14:07:31 up 35 min,  1 user,  load average: 0.02, 0.04, 0.00
Tasks: 22 total,  1 running, 21 sleeping,  0 stopped,  0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem : 7754.6 total, 7142.6 free,  532.0 used,  230.3 buff/cache
MiB Swap: 2048.0 total, 2048.0 free,  0.0 used,  7222.6 avail Mem

  PID USER      PR  NI  VIRT  RES  SHR S %CPU  %MEM    TIME+  COMMAND
 837 teja      20   0   9300 5700 3512 R  0.3   0.1   0:00.05 top
   1 root      20   0  21860 13148 9800 S  0.0   0.2   0:01.69 systemd
   2 root      20   0   3180 2204 2072 S  0.0   0.0   0:00.03 init-systemd(Ub
   6 root      20   0   3196 2128 2008 S  0.0   0.0   0:00.00 init
   58 root     19  -1 50368 16184 15100 S  0.0   0.2   0:00.49 systemd-journal
 109 root     20   0 25284 6712 5176 S  0.0   0.1   0:01.18 systemd-udev
 123 systemd+ 20   0 21344 13244 11120 S  0.0   0.2   0:00.28 systemd-resolve
 124 systemd+ 20   0 91036 8000 7032 S  0.0   0.1   0:00.37 systemd-timesyn
 187 root     20   0  4244 2680 2432 S  0.0   0.0   0:00.02 cron
 188 message+ 20   0  9548 5344 4640 S  0.0   0.1   0:00.17 dbus-daemon
 202 root     20   0 17980 8808 7764 S  0.0   0.1   0:00.17 systemd-logind
 226 root     20   0  3124 1968 1836 S  0.0   0.0   0:00.01 agetty
 227 syslog    20   0 222516 5964 4568 S  0.0   0.1   0:00.19 rsyslogd
 234 root     20   0 107036 23256 13804 S  0.0   0.3   0:00.32 unattended-upgr
 340 root     20   0  6704 4656 3872 S  0.0   0.1   0:00.03 login
 390 teja     20   0 20324 11504 9376 S  0.0   0.1   0:00.26 systemd
 391 teja     20   0 21168 3596 1868 S  0.0   0.0   0:00.00 (sd-pam)
 415 teja     20   0  6080 5360 3696 S  0.0   0.1   0:00.05 bash
 700 root     20   0  3184 1116  980 S  0.0   0.0   0:00.00 SessionLeader
 702 root     20   0  3200 1256 1108 S  0.0   0.0   0:00.02 Relay(705)
 705 teja     20   0  6080 5376 3656 S  0.0   0.1   0:00.09 bash
 793 root     20   0 1830360 14308 11768 S  0.0   0.2   0:00.33 wsl-pro-service

```

The `fork()` system call is used to create a new process. The original process is called the **parent process**, and the newly created process is called the **child process**. Each process has its own **PID**, and the child process can get its parent's ID using **PPID**. During execution, processes move through different states such as running, waiting, and terminated.